

Practical-03

Develop a C program using fork() that creates a parent and child process. Display the Process ID (PID), Parent Process ID (PPID), and process states at different stages of execution. Design an experiment to observe process state transitions (Ready, Running, Waiting, Terminated) using Linux monitoring tools such as ps, top, and /proc. Document the observations.

OUTPUT :

```
v_santosh@DESKTOP-E34M7RI:~/process_experiment$ pwd
/home/v_santosh/process_experiment
v_santosh@DESKTOP-E34M7RI:~/process_experiment$ nano process_states.c
v_santosh@DESKTOP-E34M7RI:~/process_experiment$ gcc process_states.c -o process_states
v_santosh@DESKTOP-E34M7RI:~/process_experiment$ ls -l
total 20
-rwxr-xr-x 1 v_santosh v_santosh 16296 Aug  8 08:59 process_states
-rw-r--r-- 1 v_santosh v_santosh 1185 Aug  8 08:59 process_states.c
v_santosh@DESKTOP-E34M7RI:~/process_experiment$ ./process_states
Parent process started
Parent PID : 1023
Parent PPID : 921

--- PARENT PROCESS ---
Parent PID : 1023
Parent PPID : 921
Parent state: RUNNING
Parent waiting for child...

--- CHILD PROCESS ---
Child PID : 1024
Child PPID : 1023
Child state: RUNNING
Child entering WAITING state...
Child resumed - RUNNING
Child terminating...
Child terminated.
Parent resumed - RUNNING
Parent terminating...
v_santosh@DESKTOP-E34M7RI:~/process_experiment$ |
```

```

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main() {
    pid_t pid;

    printf("Parent process started\n");
    printf("Parent PID : %d\n", getpid());
    printf("Parent PPID : %d\n", getppid());

    pid = fork();

    if (pid < 0) {
        perror("fork failed");
        return 1;
    }

    if (pid == 0) {
        printf("\n--- CHILD PROCESS ---\n");
        printf("Child PID : %d\n", getpid());
        printf("Child PPID : %d\n", getppid());
        printf("Child state: RUNNING\n");

        printf("Child entering WAITING state...\n");
        sleep(10);

        printf("Child resumed - RUNNING\n");

        sleep(5);

        printf("Child terminating...\n");
        exit(0);
    }
    else {
        printf("\n--- PARENT PROCESS ---\n");
        printf("Parent PID : %d\n", getpid());
        printf("Parent PPID : %d\n", getppid());
        printf("Parent state: RUNNING\n");

        printf("Parent waiting for child...\n");

        wait(NULL);

        printf("Child terminated.\n");
        printf("Parent resumed - RUNNING\n");

        printf("Parent terminating...\n");
    }

    return 0;
}

```